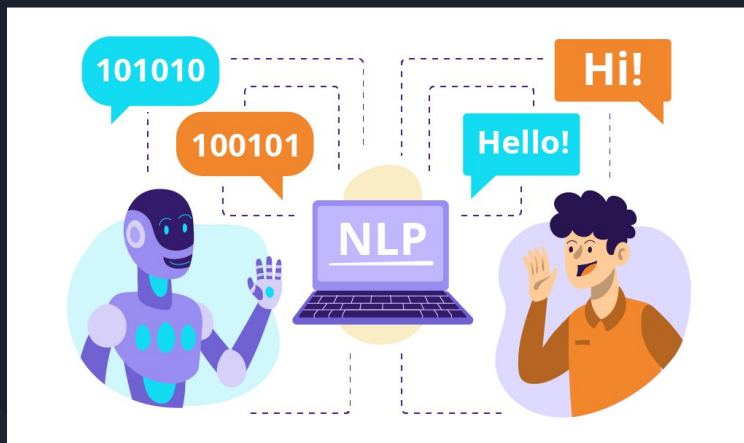
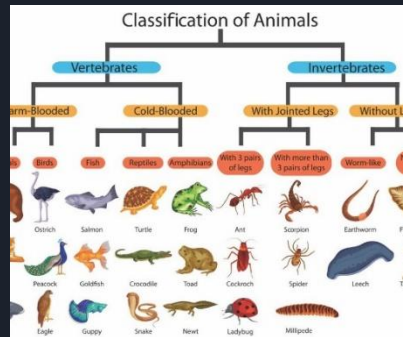


# NLP introduction



# Last lecture agenda

- Python basics:
  - Functions
  - Object oriented
  - The Zen of Python
  - Files handling



*NumPy*

- Data engineering with Python
  - NumPy fundamentals
  - Pandas fundamentals



# Reminder Exercise



- Calculate the number of female passengers on the Titanic.
- Calculate the number of male passengers on the Titanic.
- Calculate the number of minor passengers of the Titanic, all passengers that are younger than 18 years.
- Find the percent of the survived females from all-female passengers.
- Calculate the percent of the survived males from all-male passengers.
- Calculate the percent of the survived minors from all minor passengers.

# Solution



```
1 all_women = len(titanic_df.loc[(titanic_df.sex == 'female'), 'name'])
2 all_men = len(titanic_df.loc[(titanic_df.sex == 'male'), 'name'])
3 print("The number of the female passengers:", all_women)
4 print("The number of the male passengers:", all_men)
```

The number of the female passengers: 466

The number of the male passengers: 843

```
1 survived_women = len(titanic_df.loc[(titanic_df.sex == 'female') & (titanic_df.survived == 1), 'name'])
2 dead_women = len(titanic_df.loc[(titanic_df.sex == 'female') & (titanic_df.survived == 0), 'name'])
3 women_percent = survived_women/all_women * 100
4 women_percent
```

72.74678111587983

```
1 survived_men = len(titanic_df.loc[(titanic_df.sex == 'male') & (titanic_df.survived == 1), 'name'])
2 dead_men = len(titanic_df.loc[(titanic_df.sex == 'male') & (titanic_df.survived == 0), 'name'])
3 men_percent = survived_men/all_men * 100
4 men_percent
```

19.098457888493474

```
1 all_minor = len(titanic_df.loc[(titanic_df.age < 18) , 'name'])
2 all_minor
```

154

```
1 survived_minor = len(titanic_df.loc[(titanic_df.age < 18) & (titanic_df.survived == 1), 'name'])
2 dead_minor = len(titanic_df.loc[(titanic_df.age < 18) & (titanic_df.survived == 0), 'name'])
3 minor_percent = survived_minor/all_minor * 100
4 minor_percent
```

52.5974025974026

# lecture agenda

- Text Preprocessing (Tokenization, Stopword Removal, Lemmatization)
- Word Embeddings (Word2Vec, GloVe, FastText)
- Named Entity Recognition (SpaCy, HuggingFace Transformers)
- Text Classification and Sequence Labeling
- Sentence Similarity and Embedding Models (SBERT, USE)

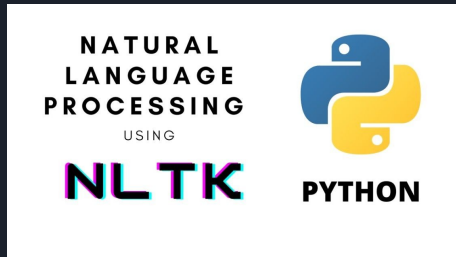
# NLTK

By the end of this session, participants will be able to:

1. Break raw text into tokens (words, punctuation).
2. Remove common “stop” words that carry little meaning.
3. Convert words to their base forms (lemmas) for consistency.

`pip install nltk spacy`

4. Download NLTK data: `python -m nltk.downloader punkt stopwords wordnet`
5. Download spaCy model: `python -m spacy download en_core_web_sm`



# NLTK basic example

- Brings in the Natural Language Toolkit library so you can call its functions.
- NLTK's tokenizers rely on pre-trained models stored under the “punkt\_tab” resource.
- This line checks your local nltk\_data folder; if “punkt\_tab” isn't present, it downloads and unzips the necessary files.
- You only need to run this once per environment.

```
nltk_basic_example.py ×  
1 import nltk  
2  
3 # Download the new tokenizer models  
4 nltk.download('punkt_tab')  
5  
6 from nltk.tokenize import word_tokenize  
7  
8 text = "Natural Language Processing with Python is fun!"  
9 tokens = word_tokenize(text)  
10 print(tokens)
```

```
C:\Users\alexx\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Use  
[nltk_data] Downloading package punkt_tab to  
[nltk_data] C:\Users\alexx\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping tokenizers\punkt_tab.zip.  
['Natural', 'Language', 'Processing', 'with', 'Python', 'is', 'fun', '!']  
  
Process finished with exit code 0
```

# NLTK VS splitting

- `split()` keeps punctuation attached and mishandles contractions; `word_tokenize` separates them intelligently.
- Shows practical benefit: downstream models get clean tokens.

```
nltk_vs_naive_splitting.py x
1 from nltk.tokenize import word_tokenize
2 import nltk
3 nltk.download('punkt', quiet=True)
4
5 text = "Don't stop believing! It's amazing."
6
7 print("split():", text.split())
8 print("word_tokenize():", word_tokenize(text))
```

```
nl nltk_vs_naive_splitting x
C:\Users\alexk\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alexk\Py
split(): ["Don't", 'stop', 'believing!', "It's", 'amazing.']
word_tokenize(): ['Do', 'n't', 'stop', 'believing', '!', 'It', 's', 'amazing', '.']

Process finished with exit code 0
```



# Word Frequency / Bag of Words (BoW)

- Tokenization splits into words and punctuation.
- Lowercasing normalizes.
- Bag of Words representation: text  $\rightarrow$  frequency vector.
- Foundation for simple classification (e.g., spam vs. not-spam) or similarity.

```
nlk_word_frequency.py x
1 from nltk.tokenize import word_tokenize
2 from collections import Counter
3 import nltk
4 nltk.download('punkt', quiet=True)
5
6 text = "Python is great. Python is simple. NLP with Python is powerful!"
7 tokens = [t.lower() for t in word_tokenize(text)]
8 freq = Counter(tokens)
9
10 print("Tokens:", tokens)
11 print("Frequencies:", freq)
```

```
nlk_word_frequency x
:
C:\Users\alexx\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alexx\PycharmProjects\AI_project3\nltk-1\nlk_word_frequency.py
Tokens: ['python', 'is', 'great', '.', 'python', 'is', 'simple', '.', 'nlp', 'with', 'python', 'is', 'powerful', '!']
Frequencies: Counter({'python': 3, 'is': 3, '.': 2, 'great': 1, 'simple': 1, 'nlp': 1, 'with': 1, 'powerful': 1, '!': 1})

Process finished with exit code 0
```

# Stopword Removal + Token Filtering

- After tokenization you can clean tokens: remove punctuation (isalpha()), remove common filler words (“the”, “is”) that dilute signal.
- Preprocessing pipeline step before vectorization or modeling.

stopwords\_example.py ×

```
1 from nltk.tokenize import word_tokenize
2 from nltk.corpus import stopwords
3 import nltk
4 nltk.download('punkt', quiet=True)
5 nltk.download('stopwords', quiet=True)
6
7 text = "This is an example showing how tokenization and stopword removal work."
8 tokens = [t.lower() for t in word_tokenize(text)]
9 filtered = [t for t in tokens if t.isalpha() and t not in stopwords.words('english')]
10
11 print("Original tokens:", tokens)
12 print("Filtered tokens:", filtered)
```

```
C:\Users\alexx\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alexx\PycharmProjects\AI_project3\nltk-1\stopwords_example.py
Original tokens: ['this', 'is', 'an', 'example', 'showing', 'how', 'tokenization', 'and', 'stopword', 'removal', 'work', '.']
Filtered tokens: ['example', 'showing', 'tokenization', 'stopword', 'removal', 'work']
```

```
Process finished with exit code 0
```

# Basic Sentiment Heuristic using Token Presence

- Tokenization enables rule-based sentiment heuristics.
- Introduce the idea of lexicons and how presence/absence of tokens changes judgment.

```
sentiment_example.py x
1 from nltk.tokenize import word_tokenize
2 import nltk
3 nltk.download('punkt', quiet=True)
4
5 positive = {"good", "great", "happy", "fun", "love", "powerful"}
6 negative = {"bad", "sad", "hate", "terrible", "hard"}
7
8 2 usages
9 def sentiment(text):
10     tokens = [t.lower() for t in word_tokenize(text)]
11     pos = sum(1 for t in tokens if t in positive)
12     neg = sum(1 for t in tokens if t in negative)
13     if pos > neg:
14         return "Positive"
15     elif neg > pos:
16         return "Negative"
17     else:
18         return "Neutral"
19
20 print(sentiment("I love using Python for NLP. It's fun and powerful!"))
21 print(sentiment("This is a terrible example, I hate bugs."))
```

```
C:\Users\alexk\PycharmProjects\AI_proj
Positive
Negative

Process finished with exit code 0
```

# REGEX based tokenization

- By default word\_tokenize splits on punctuation, but often you want to treat things like “aren’t” or em-dashes more gracefully.
- We use a regex `\w+` to pull out only letters and digits.
- Contractions become split (“aren” + “t”), which you may or may not prefer.

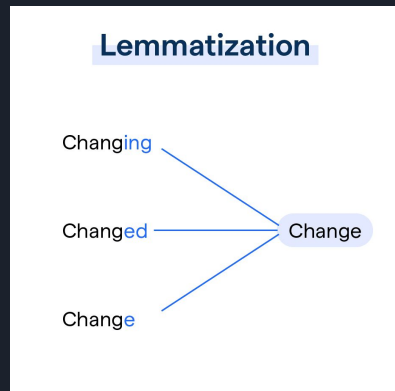
```
1 from nltk.tokenize import RegexpTokenizer
2
3 text = "Children aren't playing soccer—they've gone inside."
4
5 # \w+ matches alphanumeric "words" only (drops punctuation entirely)
6 simple_tok = RegexpTokenizer(r'\w+')
7 tokens = simple_tok.tokenize(text)
8 print(tokens)
9 # → ['Children', 'aren', 't', 'playing', 'soccer', 'they', 've', 'gone', 'inside']
```

# What is Lemmatization?

Lemmatization reduces words to their **dictionary (base) form**, called a *lemma*, using vocabulary and part-of-speech information.

Example:

- "running", "ran", "runs" → "run"
- "better" → "good" (context-aware)



It's smarter than simple **stemming**, which just chops endings heuristically (e.g., "running" → "run", but "better" might become "bett").

# Simple Comparison: Stemming vs Lemmatization

lemmatization\_example.py

```
1 import nltk
2 from nltk.stem import PorterStemmer, WordNetLemmatizer
3 from nltk import word_tokenize, pos_tag
4 from nltk.corpus import wordnet
5
6 # Downloads (run once)
7 nltk.download('punkt', quiet=True)
8 nltk.download('wordnet', quiet=True)
9 nltk.download('omw-1.4', quiet=True)
10 nltk.download('averaged_perceptron_tagger', quiet=True)
11
12 # Helpers to map NLTK POS tags to WordNet's format
13 def get_wordnet_pos(treebank_tag):
14     if treebank_tag.startswith('J'):
15         return wordnet.ADJ
16     if treebank_tag.startswith('V'):
17         return wordnet.VERB
18     if treebank_tag.startswith('N'):
19         return wordnet.NOUN
20     if treebank_tag.startswith('R'):
21         return wordnet.ADV
22     return wordnet.NOUN # fallback
23
24 text = "The striped bats are hanging on their feet and they are better than before."
25
26 tokens = word_tokenize(text)
27 print("Tokens:", tokens)
28 pos_tags = pos_tag(tokens)
29 print("Pos Tags:", pos_tags)
30
31 stemmer = PorterStemmer()
32 lemmatizer = WordNetLemmatizer()
33
34 print("Token | Stemmed | Lemmatized")
35 for token, pos in pos_tags:
36     stem = stemmer.stem(token)
37     lemma = lemmatizer.lemmatize(token, get_wordnet_pos(pos))
38     print(f"{token:6} | {stem:7} | {lemma}")
```

⌵

```
C:\Users\alexk\PycharmProjects\AI_project3\.venv\Scripts\python.exe
Tokens: ['The', 'striped', 'bats', 'are', 'hanging', 'on', 'their', 'feet', 'and', 'they', 'are', 'better', 'than', 'before.']
Pos Tags: [('The', 'DT'), ('striped', 'JJ'), ('bats', 'NNS'), ('are', 'VBP'), ('hanging', 'VBG'), ('on', 'IN'), ('their', 'PRP'), ('feet', 'NNS'), ('and', 'CC'), ('they', 'PRP'), ('are', 'VBP'), ('better', 'JJR'), ('than', 'IN'), ('before', 'RB'), ('.', '.')]
Token | Stemmed | Lemmatized
The      | the      | The
striped  | stripe   | striped
bats     | bat      | bat
are      | are      | be
hanging  | hang     | hang
on       | on       | on
their    | their    | their
feet     | feet     | foot
and      | and      | and
they     | they     | they
are      | are      | be
better   | better   | good
than     | than     | than
before   | before   | before
.        | .        | .
```

Process finished with exit code 0

# Exercise



- Generate two general sentences.
- Perform tokenization.
- Clean the tokens (lowercase, remove punctuation)
- Counts how many words two sentence have in common.
- Calculate the percentage overlap.

# Solution



nltk\_exercise.py x

```
5 nltk.download('punkt', quiet=True)
6
7 6 usages
8 def tokenize(text):
9     return [t.lower() for t in word_tokenize(text) if t.isalpha()]
10
11 1 usage
12 def common_word_count(s1, s2):
13     return len(set(tokenize(s1)) & set(tokenize(s2)))
14
15 1 usage
16 def percentage_overlap(s1, s2):
17     t1 = set(tokenize(s1))
18     t2 = set(tokenize(s2))
19     if not (t1 | t2):
20         return 0.0
21     return len(t1 & t2) / len(t1 | t2) * 100.0
22
23 # Example
24 a = "Natural language processing is fun!"
25 b = "I find processing natural language quite enjoyable."
26
27 print("Tokens A:", tokenize(a))
28 print("Tokens B:", tokenize(b))
29 print("Shared count:", common_word_count(a, b))
30 print(f"Overlap %: {percentage_overlap(a, b):.1f}%")
```

nltk\_exercise x

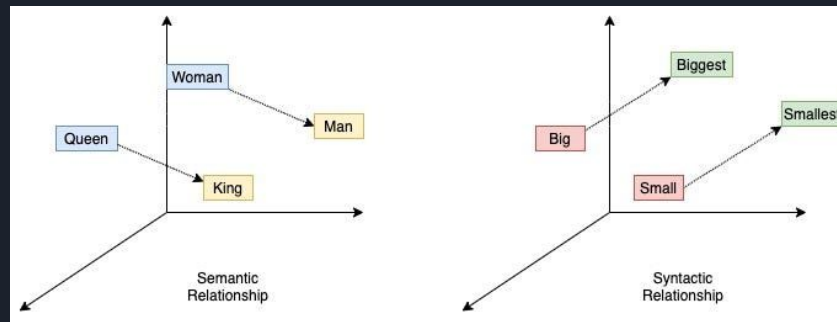
```
C:\Users\alexk\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alexk\F
Tokens A: ['natural', 'language', 'processing', 'is', 'fun']
Tokens B: ['i', 'find', 'processing', 'natural', 'language', 'quite', 'enjoyable']
Shared count: 3
Overlap %: 33.3%

Process finished with exit code 0
```



# Word2Vec with Gensim

- **Skip-gram**: predict context words from a target word.
- **CBOW**: predict a target word from surrounding context.
- Learns vectors so that words appearing in similar contexts end up close in space.
- “king” – “man” + “woman”  $\approx$  “queen”
- `pip install gensim numpy`



# Word2Vec with Gensim

```
1 import gensim.downloader as api
2
3 # this is a one-time download + cache
4 model = api.load("glove-wiki-gigaword-50")
5
6 print("king ~ queen:", model.similarity("king", "queen"))
7 print("paris + germany - france →",
8       [w for w, _ in model.most_similar(
9           positive=["paris", "germany"],
10          negative=["france"],
11          topn=3)])
```

```
C:\Users\alexk\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alexk\PycharmProjects\AI_project3\scripts\word2vec.py
[=====] 100.0% 66.0/66.0MB downloaded
king ~ queen: 0.7839043
paris + germany - france → ['berlin', 'frankfurt', 'vienna']
```

```
Process finished with exit code 0
```

# Word2Vec with Gensim

Loads a pre-trained GloVe embedding model (“glove-wiki-gigaword-50”).

- “50” means each word is represented as a 50-dimensional vector.
- The first time you run this it downloads the model; afterward it’s cached locally.
- These vectors were trained on Wikipedia + Gigaword corpus to capture word meaning by context.
- `print("king ~ queen:", model.similarity("king", "queen"))`

# Word2Vec with Gensim

Computes the **cosine similarity** between the vectors for "king" and "queen".

- Cosine similarity is a number between -1 and 1 that measures how close the two vectors point in direction; higher means more semantically similar.
- So this prints something like:  
king ~ queen: 0.73 (actual value may vary slightly)
- Interpretation: "king" and "queen" are related in meaning, so their vectors are close.

# Word2Vec with Gensim

- This does a **vector analogy** operation.
  - a. It computes: `vec("paris") + vec("germany") - vec("france")` and finds the top 3 words whose vectors are closest to that result.
  - b. Intuition: if “Paris is to France” as “X is to Germany”, then X should come out—expected answer is something like "Berlin".
  - c. So you might see output like:  
`paris + germany - france` → `['berlin', 'munich', ...]` depending on nearest neighbors.

## Why this works

Word embeddings place words with similar contexts near each other in space. Analogies exploit linear relationships in that space. For example:

$$\text{vec}(\text{"king"}) - \text{vec}(\text{"man"}) + \text{vec}(\text{"woman"}) \approx \text{vec}(\text{"queen"})$$

# Sentence Similarity & Semantic Search

- Find words similarity

```
similarity.py x
1 from gensim.models import Word2Vec
2 from nltk.tokenize import word_tokenize
3 import nltk
4
5 nltk.download('punkt', quiet=True)
6
7 sentences = [
8     "I love natural language processing",
9     "Language models capture semantic meaning",
10    "Word embeddings help with similarity tasks",
11    "Deep learning creates vector representations of words",
12    "We can find similar words using Word2Vec"
13 ]
14 tokenized = [word_tokenize(s.lower()) for s in sentences]
15
16 model = Word2Vec(tokenized, vector_size=50, window=2, min_count=1, sg=1)
17
18 print("Words most similar to 'language':", model.wv.most_similar("language", topn=3))
19 print("Words most similar to 'deep':", model.wv.most_similar("deep", topn=3))
20
```

```
similarity x
C:\Users\alekx\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alekx\PycharmProjects\AI_project3\word2vec-2\similarity.py
Words most similar to 'language': [('learning', 0.2706388533115387), ('similarity', 0.21057100594043732), ('we', 0.18602196872234344)]
Words most similar to 'deep': [('find', 0.2764217257499695), ('natural', 0.27047690749168396), ('processing', 0.20483414828777313)]

Process finished with exit code 0
```

# Synonym Ranking via WordNet + GloVe

- Uses NLTK's WordNet to fetch candidate synonyms
- Ranks them by GloVe cosine similarity to the original word

```
1 import nltk
2 from nltk.corpus import wordnet
3 from gensim.downloader import load
4 from scipy.spatial.distance import cosine
5
6 nltk.download('wordnet')
7 model = load("glove-wiki-gigaword-50")
8
9 1 usage
10 def top_synonyms(word, n=5):
11     # gather lemmas of all synsets
12     lemmas = set(
13         l.name().replace('_', ' ')
14         for syn in wordnet.synsets(word)
15         for l in syn.lemmas()
16         if l.name().lower() != word.lower()
17     )
18     # filter to those in the model
19     candidates = [w for w in lemmas if w in model]
20     # score by similarity
21     scored = sorted(
22         [(w, 1 - cosine(model[word], model[w])) for w in candidates],
23         key=lambda x: -x[1]
24     )
25     return scored[:n]
26
27 print("Top synonyms for 'happy':", top_synonyms("happy"))
```

```
C:\Users\alexk\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alexk\PycharmProjects\AI_project3\word2vec-2\synonym.py
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\alexk\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Top synonyms for 'happy': [('glad', 0.865877219251113), ('felicitous', 0.05687176712781572), ('well-chosen', -0.1973035191118635)]
```

```
Process finished with exit code 0
```

# Fast text example

- This is example of FastText usage:

```
fast_text_example.py x
1 from gensim.models import Word2Vec, FastText
2 from nltk.tokenize import word_tokenize
3 import nltk
4
5 nltk.download('punkt', quiet=True)
6
7 # Tiny corpus
8 sentences = [
9     "I like playing soccer",
10    "He enjoys playing football",
11    "She loves sports and games",
12 ]
13
14 tokenized = [word_tokenize(s.lower()) for s in sentences]
15
16 # Train Word2Vec and FastText on same data
17 w2v = Word2Vec(tokenized, vector_size=20, window=2, min_count=1, sg=1)
18 ft = FastText(tokenized, vector_size=20, window=2, min_count=1, sg=1)
19
20 # Word that was seen
21 print("Word2Vec similarity between 'playing' and 'soccer':",
22       w2v.wv.similarity("playing", "soccer"))
23 print("FastText similarity between 'playing' and 'soccer':",
24       ft.wv.similarity("playing", "soccer"))
25
26 # Slightly misspelled / unseen word
27 misspelled = "playng" # missing 'i'
28 print("\nWord2Vec knows 'playng'?", misspelled in w2v.wv.key_to_index)
29 try:
30     print("Word2Vec similarity 'playng' vs 'playing':",
31           w2v.wv.similarity(misspelled, "playing"))
32 except KeyError:
33     print("Word2Vec: 'playng' is out of vocabulary (no vector)")
34
35 print("FastText knows 'playng' by subwords and similarity to 'playing':",
36       ft.wv.similarity(misspelled, "playing"))
37
```

```
C:\Users\alexk\PycharmProjects\AI_project3\.venv\Scripts\python.exe C:\Users\alexk\PycharmProjects\AI_project3\fast_text_example.py
Word2Vec similarity between 'playing' and 'soccer': -0.16369319
FastText similarity between 'playing' and 'soccer': -0.4168487

Word2Vec knows 'playng'? False
Word2Vec: 'playng' is out of vocabulary (no vector)
FastText knows 'playng' by subwords and similarity to 'playing': 0.62483734

Process finished with exit code 0
```



# Fast text example

Imagine words are toys

You have two toy boxes: one called **Word2Vec** and one called **FastText**.

**Word2Vec box:**

- It only knows whole toys it was shown before.
- If you give it a new toy that looks almost the same but is missing a piece (“playng” instead of “playing”), it says: “I don’t know that one.” (That’s why it failed.)

**FastText box:**

- It doesn’t just remember whole toys. It also knows the little pieces the toys are made of (like LEGO bricks).
- So if you give it “playng” (missing an “i”), it sees the same pieces as “playing” and says: “Oh, that’s close!” — that’s why similarity is high (0.62).

# Fast text example

## What about “playing” and “soccer”?

You asked both boxes: “Are ‘playing’ and ‘soccer’ similar?”

- They are kind of related (both are about games/sports), but in this tiny toy example they weren’t used together enough, so the boxes don’t think they’re very close.
- The **negative numbers** (like -0.16 or -0.41) just mean “not similar” in their internal math — they’re not best friends. If the number were big and positive, that would mean “these words are close buddies.”

## Summary:

- **Word2Vec**: knows exact words only. “playng” is unknown.
- **FastText**: builds words from pieces, so even a slightly broken word (“playng”) still makes sense.
- “playing” vs “soccer” isn’t very close in this small example, so similarity is low (even negative).



# Exercise



- These are your three sentences:
  - "I love NLP."
  - "Natural language processing is fun."
  - "Learning about language helps."
- Preprocess each sentence:
  - Tokenize
  - Lowercase
  - Remove English stopwords
  - Lemmatize
- Train **Word2Vec** on the preprocessed corpus.
- Train **FastText** on the same preprocessed corpus.

## Exercise



- For the word "language": print top 2 most similar words from both models.

For the misspelled word "langauge" (typo of "language"):

- Try similarity to "language" with both models.
- Note: Word2Vec will fail; FastText will give a score

# Solution



```
exercise.py x
1 import nltk
2 from nltk.tokenize import word_tokenize
3 from nltk.corpus import stopwords, wordnet
4 from nltk import pos_tag
5 from nltk.stem import WordNetLemmatizer
6 from gensim.models import Word2Vec, FastText
7
8 # minimal downloads
9 nltk.download('punkt', quiet=True)
10 nltk.download('stopwords', quiet=True)
11 nltk.download('wordnet', quiet=True)
12 nltk.download('averaged_perceptron_tagger', quiet=True)
13
14 STOP = set(stopwords.words('english'))
15 lemmatizer = WordNetLemmatizer()
16
17 1 usage
18 def wn_pos(tag):
19     return {
20         'J': wordnet.ADJ,
21         'V': wordnet.VERB,
22         'N': wordnet.NOUN,
23         'R': wordnet.ADV
24     }.get(tag[0], wordnet.NOUN)
25
26 1 usage
27 def preprocess(s):
28     tokens = word_tokenize(s.lower())
29     tagged = pos_tag(tokens)
30     final_preprocess = [
31         lemmatizer.lemmatize(t, wn_pos(p))
32         for t, p in tagged
33         if t.isalpha() and t not in STOP
34     ]
35     print("Final preprocess:", final_preprocess)
36     return final_preprocess
```

```
36 corpus = [
37     "I love NLP.",
38     "Natural language processing is fun.",
39     "Learning about language helps."
40 ]
41 docs = [preprocess(s) for s in corpus]
42
43 # Train embeddings
44 w2v = Word2Vec(docs, vector_size=30, window=2, min_count=1, sg=1)
45 ft = FastText(docs, vector_size=30, window=2, min_count=1, sg=1)
46
47 # Similar to "language"
48 print("Word2Vec similar to 'language':", w2v.wv.most_similar("language", topn=2))
49 print("FastText similar to 'language':", ft.wv.most_similar("language", topn=2))
50
51 # Typo case
52 typo = "langauge"
53 print("\nWord2Vec has typo?", typo in w2v.wv.key_to_index)
54 try:
55     print("Word2Vec similarity typo->language:", w2v.wv.similarity(typo, "language"))
56 except KeyError:
57     print("Word2Vec: typo out of vocab")
58
59 print("FastText similarity typo->language:", ft.wv.similarity(typo, "language"))
60
```

Final preprocess: ['love', 'nlp']

Final preprocess: ['natural', 'language', 'processing', 'fun']

Final preprocess: ['learn', 'language', 'help']

Word2Vec similar to 'language': [('love', 0.05847087875008583), ('learn', 0.003974515944719315)]

FastText similar to 'language': [('help', 0.17071162164211273), ('natural', 0.11909386794404984)]

Word2Vec has typo? False

Word2Vec: typo out of vocab

FastText similarity typo->language: 0.07028107

# Spacy

spaCy is a fast, production-grade Python library for NLP. It gives you ready-made pipelines for:

- Tokenization
- Part-of-speech tagging
- Lemmatization
- Named entity recognition (NER)
- Dependency parsing
- (With the right model) word vectors / similarity

It hides a lot of the plumbing so you can get meaningful structure out of raw text with just a few lines.

# Spacy

```
spacy_example.p ×
1 import spacy
2
3 # Load the small English model
4 nlp = spacy.load("en_core_web_sm")
5
6 text = "Apple is looking at buying U.K. startup for $1 billion. I love processing text with"
7
8 doc = nlp(text) # run the pipeline
9
10 # Tokens, lemmas, POS
11 print("Token | Lemma | POS")
12 for token in doc:
13     print(f"{token.text:8} | {token.lemma_:8} | {token.pos_}")
14
15 # Named Entities
16 print("\nNamed Entities:")
17 for ent in doc.ents:
18     print(f"{ent.text} ({ent.label_})")
19
20 # Simple similarity (requires vectors; small model has limited vectors but still works)
21 sent1 = nlp("I enjoy natural language processing.")
22 sent2 = nlp("I like working with text.")
23 print("\nSimilarity between sentences:", round(sent1.similarity(sent2), 3))
24
```

# Spacy name entity example

```
spacy_name_entity.py x
1 import spacy
2 nlp = spacy.load("en_core_web_sm")
3 doc = nlp("Apple is buying a startup in the U.K. for $1 billion.")
4
5 for ent in doc.ents:
6     print(ent.text, "→", ent.label_)
7
```

```
C:\Users\alexk\PycharmProjects\AI_pr
Apple → ORG
U.K. → GPE
$1 billion → MONEY

Process finished with exit code 0
```



# Hugging face

Hugging Face is both a library and a **community-driven hub** for machine learning models and datasets. People around the world:

- **Publish pretrained models** (NLP, vision, audio, multimodal) so others can reuse them instantly.
- **Share datasets** in standardized formats.
- Build **Spaces**: lightweight web apps (usually Streamlit or Gradio) to demo models interactively.
- Collaborate on fine-tuning, evaluation, and extending models (open sharing, versioning, and metadata).

It's like an app store + social workspace for ML: you don't need to train from scratch—you can browse the hub, pick a model, and use it in minutes.



# Hugging Face examples

Here are examples of few interesting models.

**Sentiment analysis (default “trusted” model, e.g., distilbert-base-uncased-finetuned-sst-2-english)**

<https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english>

**Fill-mask model (distilbert-base-uncased)**

<https://huggingface.co/distilbert-base-uncased>

**Summarization (facebook/bart-large-cnn)**

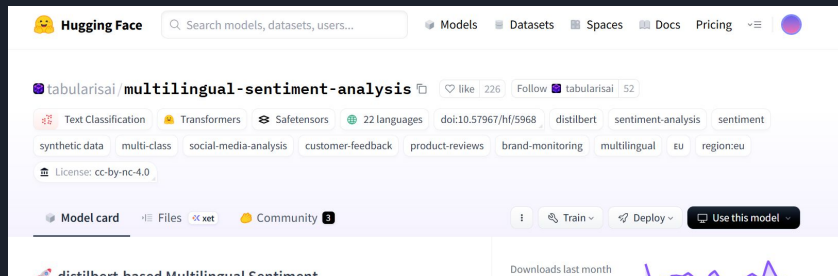
<https://huggingface.co/facebook/bart-large-cnn>

**Sentence embeddings (sentence-transformers/all-MiniLM-L6-v2)**

<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

**Image captioning / multimodal (nlpconnect/vit-gpt2-image-captioning)**

<https://huggingface.co/nlpconnect/vit-gpt2-image-captioning>



# Exercise



- Find a fitting Translation model on Hugging face and validate that you are able to execute it.

# Thank you for your time

